Task 3: Building a Deep Neural Network from Scratch

Part A — NumPy from scratch · Part B — The same network in PyTorch

---

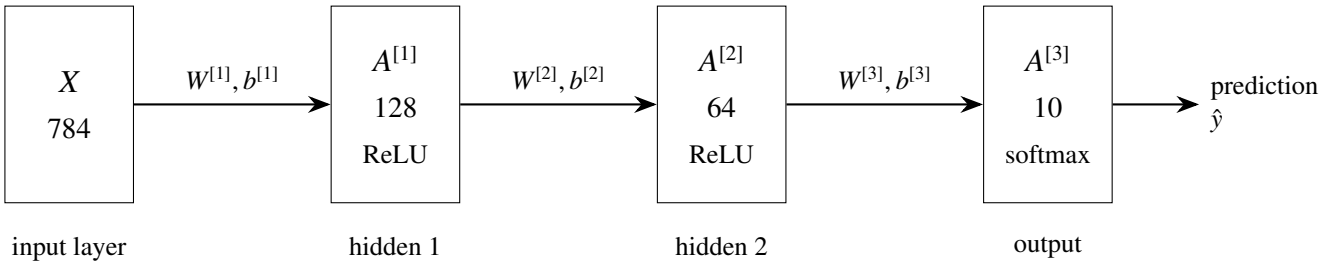
1. What you are building

You will implement a fully connected neural network for MNIST digit classification twice.

Part A — entirely in NumPy. No PyTorch, no TensorFlow, no autograd. Every gradient in this network will be computed by code you wrote.

Part B — the same architecture in PyTorch, then a comparison of the two.

The architecture:




```
layer_dims = [784, 128, 64, 10]
```

Your code must **not** hardcode three layers. It takes `layer_dims` as an argument and works for any depth — `[784, 10]` or `[784, 512, 256, 128, 10]` must run without changing a single line.

Why this matters

> Writing it for a fixed depth is easier and teaches you almost nothing. Writing it as a loop over layers forces you to see that a deep network is one operation repeated, and that backpropagation is one rule applied recursively. That is the entire idea of this task.

2. Notation and shapes

We put **examples in columns**. If m is the number of examples in a batch:

Symbol	Meaning	Shape
$X = A^{[0]}$	input batch	$(784, m)$
$W^{[l]}$	weights of layer l	$(n_l,\ n_{l-1})$
$b^{[l]}$	biases of layer l	$(n_l,\ 1)$
$Z^{[l]}$	pre-activation	$(n_l,\ m)$
$A^{[l]}$	activation	$(n_l,\ m)$
Y	one-hot true labels	$(10, m)$
$A^{[L]}$	network output (probabilities)	$(10, m)$

Here $n_0 = 784$, $n_1 = 128$, $n_2 = 64$, $n_3 = 10$, and $L = 3$.

Check your shapes constantly

Most of the time you will lose on this task will be spent on shape errors. Print `.shape` after every operation while developing. A useful habit: write an `assert` after each computation, e.g. `assert Z.shape == (W.shape[0], A_prev.shape[1])`. A wrong shape that happens to broadcast silently is far worse than a crash.

3. Step 1 — Initialization

What to write

```
def initialize_parameters(layer_dims, seed=0):
    """
    layer_dims : list of ints, e.g. [784, 128, 64, 10]
    returns    : dict with keys "W1","b1","W2","b2", ...
    """
```

Loop over $l = 1 \dots L$ and create:

$$W^{[l]} \sim \mathcal{N}\left(0,\ \frac{2}{n_{l-1}}\right), \qquad b^{[l]} = \mathbf{0}$$

In code, that is `np.random.randn(n_l, n_prev) * np.sqrt(2.0 / n_prev)`.

Why not initialize all weights to zero?

Try it and watch what happens: the network refuses to learn.
If every weight in a layer is zero (or any single identical value), then every neuron in that layer computes exactly the same z, produces exactly the same activation, and — when the gradient flows back — receives exactly the same dW. They start identical and they update identically, forever. A layer of 128 neurons behaves as a single neuron.

This is called the **symmetry problem**. Random initialization exists to break it. Note that biases *can* safely be zero, because the weights are already random — the symmetry is broken by W alone.

Why not just use large random weights?

Suppose you use `np.random.randn(...) * 1.0`. Each z is a sum of 784 terms, so z ends up very large in magnitude. Run that through several layers and the activations explode — and with a softmax at the end you get overflow and `NaN`.
Now try the opposite, `* 0.01`. Each layer shrinks the signal, and after three layers the activations are almost zero. The gradients that flow back are correspondingly tiny, and training crawls.
So the scale matters, and it is not a free parameter you guess. **He initialization** answers it: for a layer with n_{l-1} inputs and a ReLU activation, drawing weights with variance $2/n_{l-1}$ keeps the variance of the activations roughly constant as the signal moves forward through the layers. The factor of 2 is there specifically because ReLU zeroes out half its inputs, so it destroys half the variance and you have to compensate.
Experiment worth doing (2 minutes, and it will stay with you): initialize with `* 0.01` instead, train for one epoch, and compare the loss curve to He. Then try `* 1.0`. Report what you saw.
Reference: He et al., *Delving Deep into Rectifiers* (2015), `https://arxiv.org/abs/1502.01852` — section 2.2 is the relevant part. For a gentler treatment: `https://cs231n.github.io/neural-networks-2/`

4. Step 2 — Forward propagation

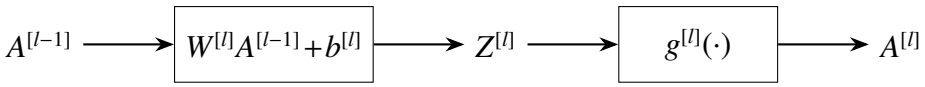
Write this as **two** functions.

2.1 One layer

```
def layer_forward(A_prev, W, b, activation):
    """
    A_prev     : (n_prev, m)
    W          : (n_l, n_prev)
    b          : (n_l, 1)
    activation : "relu" or "softmax"
    returns    : A, Z
    """
```

It computes:

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}, \qquad A^{[l]} = g^{[l]}\left(Z^{[l]}\right)$$



You return **both** A and Z. You will need Z during backpropagation and recomputing it would be wasteful — this is exactly what a framework means when it says it "stores activations for the backward pass."

The activations:

$$\text{ReLU}(z) = \max(0, z), \qquad \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Numerical stability in softmax — read this before you debug for an hour

Computing e^z directly overflows as soon as any z exceeds about 700, and you get `inf` then `NaN`. The standard fix is to subtract the maximum of each column before exponentiating:

$$\text{softmax}(z)_k = \frac{e^{z_k - \max_j z_j}}{\sum_i e^{z_i - \max_j z_j}}$$

This changes nothing mathematically — the constant cancels between numerator and denominator — but it keeps every exponent ≤ 0, so nothing overflows. Do the subtraction **per column** (`axis=0, keepdims=True`), since each column is a separate example.

2.2 The full pass

```
def model_forward(X, parameters):
    """
    returns : AL, cache
    cache holds every A and Z you will need in backprop
    """
```

Set $A^{[0]} = X$, loop $l = 1 \ldots L-1$ calling `layer_forward` with ReLU, then call it once more for layer L with softmax. Store every $A^{[l]}$ and $Z^{[l]}$ in the cache. Return $A^{[L]}$ (your predictions) and the cache.

5. Step 3 — The cost

```
def compute_cost(AL, Y):
    """
    AL : (10, m) predicted probabilities
    Y  : (10, m) one-hot labels
    returns : scalar
    """
```

Categorical cross-entropy, averaged over the batch:

$$J = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{10} Y_{k,i} \log\left(A_{k,i}^{[L]}\right)$$

Because Y is one-hot, only one term per example survives — the log-probability the network assigned to the correct class. The loss is the negative of that, averaged.

Connect this to the sheet

On the question sheet you worked with binary cross-entropy, $-[y \ln \hat{y} + (1-y)\ln(1-\hat{y})]$, and you saw what ln does as the prediction approaches the wrong answer. This is the same function generalised from 2 classes to 10. Ask yourself: what is J when the network confidently predicts the right digit? What is it when it confidently predicts a wrong one?
Practical note: add a small ε (e.g. `1e-12`) inside the log, or clip $A^{[L]}$, to avoid log(0) producing `-inf`.

6. Step 4 — Backward propagation

This is the core of the task. Again, two functions.

4.1 The starting point: $dZ^{[L]}$

Backpropagation is a loop that runs from the last layer to the first, and every layer's computation needs the gradient arriving from the layer above it. The last layer has nothing above it, so you have to start it by hand.

For softmax combined with categorical cross-entropy, the result is remarkably simple:

$$\boxed{dZ^{[L]} = A^{[L]} - Y}$$

Where this comes from — and why you are being given it

You have not derived the softmax derivative yet; we will do it in the optimization session. For now, use the result.
But notice what it says, because the meaning is available even without the derivation: the gradient at the output layer is simply **the prediction minus the truth**. If the network says 0.9 for the correct class, the gradient is −0.1 — a small push. If it says 0.05, the gradient is −0.95 — a large one. The size of the correction is exactly how wrong you were.
Compare this to Q19 on the question sheet, where binary cross-entropy plus sigmoid gave $\hat{y} - y$. It is the same result, generalised. That is not a coincidence: softmax and cross-entropy are chosen *together*, precisely so that this cancellation happens.
If you want to see the derivation now, the intuition is here: `https://www.3blue1brown.com/lessons/backpropagation-calculus`

4.2 One layer

```
def layer_backward(dA, Z, W, b, A_prev, activation):
    """
    dA     : (n_l, m)     gradient arriving from the layer above
    returns: dW, db, dA_prev
    """
```

Three steps inside:

$$\mathrm{d}Z^{[l]} = \mathrm{d}A^{[l]} * g'^{[l]}\left(Z^{[l]}\right) \qquad \text{(elementwise)} \tag{1}$$

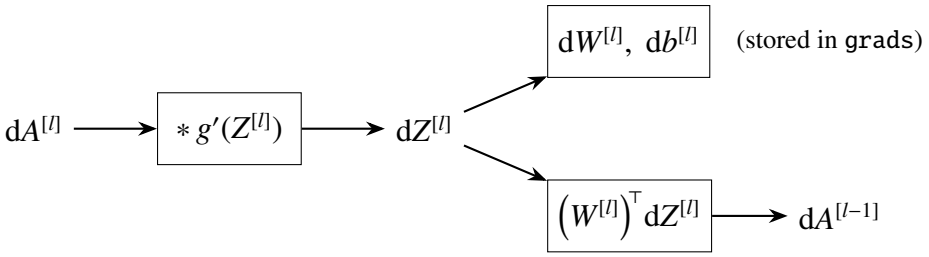
$$\mathrm{d}W^{[l]} = \frac{1}{m}\,\mathrm{d}Z^{[l]}\left(A^{[l-1]}\right)^{\top} \tag{2}$$

$$\mathrm{d}b^{[l]} = \frac{1}{m}\sum_{\text{columns}} \mathrm{d}Z^{[l]} \tag{3}$$

$$\mathrm{d}A^{[l-1]} = \left(W^{[l]}\right)^{\top}\mathrm{d}Z^{[l]} \tag{4}$$

with the ReLU derivative

$$g'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \le 0 \end{cases}$$



Read the shapes, they tell you the formula

If you forget which way a transpose goes, derive it from shapes instead of memorising. $\mathrm{d}W^{[l]}$ must have the same shape as $W^{[l]}$, which is (n_l, n_{l-1}). You have $\mathrm{d}Z^{[l]}$ of shape (n_l, m) and $A^{[l-1]}$ of shape (n_{l-1}, m). The only product that gives (n_l, n_{l-1}) is $\mathrm{d}Z^{[l]}(A^{[l-1]})^{\top}$. There is no other option.
Do the same for $\mathrm{d}A^{[l-1]}$: it must be (n_{l-1}, m), and only $(W^{[l]})^{\top}\mathrm{d}Z^{[l]}$ produces that.

Do not forget the $1/m$

$\mathrm{d}W$ and $\mathrm{d}b$ are averaged over the batch; $\mathrm{d}A^{[l-1]}$ is not. Getting this wrong is the single most common bug in this assignment, and the nastiest — your gradients come out 64 times too large, the network still trains, and the accuracy still looks reasonable. You will only catch it when you check your gradients numerically, which is Part B of this task.

4.3 The full pass

```
def model_backward(AL, Y, cache, parameters):
    """
    returns : grads, a dict with "dW1","db1","dW2","db2", ...
    """
```

Start with $\mathrm{d}Z^{[L]} = A^{[L]} - Y$ and handle layer L separately (it has no g' multiplication — that step is already folded into the boxed result). Then loop backwards $l = L-1 \ldots 1$, calling `layer_backward` and feeding each layer's $\mathrm{d}A^{[l-1]}$ into the next iteration.

The one sentence to take away

The only thing travelling backwards through the network is dA. Each layer receives it, uses it to compute its own dW and db, and hands a new dA to the layer below. That is backpropagation. Everything else is bookkeeping.

7. Step 5 — Updating the parameters

```
def update_parameters(parameters, grads, learning_rate):
    """
    returns : updated parameters dict
    """
```

$$W^{[l]} \leftarrow W^{[l]} - \eta \, \mathrm{d}W^{[l]}, \qquad b^{[l]} \leftarrow b^{[l]} - \eta \, \mathrm{d}b^{[l]}$$

This is the update rule from Q30 of the question sheet, applied to every parameter at once. The minus sign is doing the work: you move each parameter *against* its gradient, so the loss goes down rather than up.

8. Step 6 — The model class

Collect everything into one class:

```
class NeuralNetwork:
    def __init__(self, layer_dims, seed=0):
        self.parameters = initialize_parameters(layer_dims, seed)

    def forward(self, X):        ...
    def compute_cost(self, AL, Y): ...
    def backward(self, AL, Y, cache): ...
    def update(self, grads, learning_rate): ...

    def train(self, X, Y, epochs, batch_size, learning_rate): ...
    def predict(self, X):        ...
    def accuracy(self, X, Y):    ...
```

Mini-batch training

`train` does the following, for each epoch:

1. Shuffle the training examples (shuffle the *columns*, and shuffle X and Y with the same permutation — if these come apart, your labels no longer match your images and the network will never learn).
2. Split into batches of size 64. The last batch may be smaller; your code must handle that.

3. For each batch: forward → cost → backward → update.

4. Record the average cost over the epoch and print it.

Why mini-batches at all?

Two obvious alternatives exist. You could compute the gradient over all 60,000 images and take one step — that is the exact gradient of the loss you defined, but each step is expensive and you take very few of them. Or you could take a step after every single image — cheap steps, but each gradient is a noisy estimate based on one example.
A mini-batch of 64 sits between: the gradient is a good enough estimate of the true one, and you get roughly 940 updates per epoch instead of 1. If you have time, run the same network with batch size 1 and with the full 60,000 and compare the loss curves — the difference is worth seeing once.

9. Fixed specification — follow this exactly

Use these exactly. They are what make your run comparable to everyone else's, and to the reference.

```
# data
x = x.reshape(-1, 784).astype(np.float64) / 255.0
X = x.T                    # (784, m) -- examples in COLUMNS
Y = one_hot(labels)        # (10, m)

# dtype
use float64 everywhere (not float32)

# architecture
layer_dims = [784, 128, 64, 10]

# initialization -- this exact order inside the loop
np.random.seed(0)
for l in range(1, L+1):
    W[l] = np.random.randn(n_l, n_prev) * np.sqrt(2.0 / n_prev)
    b[l] = np.zeros((n_l, 1))

# training
batch_size    = 64
learning_rate = 0.1
epochs        = 10
shuffle seed  = 0
```

Why the order of the random calls matters

Every call to `np.random.randn` advances the generator's internal state. If you create $W^{[2]}$ before $W^{[1]}$, or draw the biases randomly instead of zeroing them, you get completely different weights even with the same seed — and then none of your numbers will match anyone else's.

Create $W^{[l]}$ then $b^{[l]}$, layer by layer, in order.

Part B — The same network in PyTorch

10. What to build

Rebuild the *exact same network* — `[784, 128, 64, 10]`, ReLU, softmax output, cross-entropy, mini-batch SGD — this time in PyTorch.

The point is not that PyTorch is easier. The point is that you already wrote every piece of it by hand, so you can now see precisely which of your functions each PyTorch line replaces.

```
import torch
import torch.nn as nn

class TorchNet(nn.Module):
    def __init__(self, layer_dims):
        super().__init__()
        # build the layers from layer_dims -- again, NOT hardcoded
        ...

    def forward(self, x):
        ...

model     = TorchNet([784, 128, 64, 10])
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
```

Same rule as Part A: the class takes `layer_dims` and works for any depth. Build the layers in a loop and hold them in `nn.ModuleList` or `nn.Sequential`.

Use the same hyperparameters as Part A: batch size 64, learning rate 0.1, 10 epochs.

11. Three things that will trip you up

1. Do NOT apply softmax in your forward pass

`nn.CrossEntropyLoss` expects **raw logits** — the output of your last linear layer, with no activation on it. It applies the softmax internally, together with the log, in a single numerically stable operation.

If you apply softmax yourself and then pass the result to `CrossEntropyLoss`, you are applying softmax *twice*. The code runs, no error appears, and the network trains — badly. This is the single most common PyTorch mistake and it is silent.

So your `forward` ends at the last `nn.Linear`. The softmax only appears at prediction time, if you want probabilities.

2. Labels are integers, not one-hot

In Part A you built Y as a one-hot matrix of shape $(10, m)$. `nn.CrossEntropyLoss` does not want that. It wants a 1-D tensor of integer class indices of shape $(m,)$ — just `[7, 2, 1, 0, 4, ...]`.

3. PyTorch puts examples in ROWS

This is the opposite of Part A. In your NumPy code X has shape $(784, m)$ — examples in columns. PyTorch expects $(m, 784)$ — examples in rows.
Consequently `nn.Linear(in, out)` computes $xW^\top + b$ rather than $Wx + b$. Interestingly, `layer.weight` still has shape `(out, in)` — exactly the shape of your $W^{[l]}$. So the weights match; it is the *data* layout that flips.
Be deliberate about this rather than transposing things at random until the shapes stop complaining.

12. The training loop

```
for epoch in range(epochs):
    for xb, yb in batches:
        optimizer.zero_grad()
        out  = model(xb)
        loss = criterion(out, yb)
        loss.backward()
        optimizer.step()
```

Map each line onto the code you already wrote

Answer this in writing — it is part of the submission.

- `model(xb)` corresponds to which of your functions?
- `criterion(out, yb)` corresponds to which?
- `loss.backward()` corresponds to which?
- `optimizer.step()` corresponds to which?

And one that has no counterpart in your code: **why is `optimizer.zero_grad()` needed at all?** What would happen without it? Look up what PyTorch does to `.grad` when you call `backward()` twice — and think about why your NumPy implementation never had this problem.

13. The comparison

Run both implementations under identical settings and report:

1. **Test accuracy** — yours vs PyTorch.
2. **Loss curve per epoch** — both on the same plot.
3. **Training time** — wall-clock, for the full run.

What to expect, and what it means

The accuracies should land close to each other — both around 97–98%. They will not be identical, and that is fine: the weight initialization differs (PyTorch does not use He by default for `nn.Linear`) and the shuffling differs, so the two runs follow different paths to a similar place.
The training times will *not* be close. Ask yourself why, given that both are running the same arithmetic on the same CPU. What is PyTorch doing that your loop is not?
If your accuracy is far below PyTorch's — say 90% against 97% — that is not a tuning issue. It means something in your backward pass is wrong. A network with one incorrect gradient still trains and still looks plausible; it just learns worse. This is exactly why you cannot judge a from-scratch implementation by its accuracy alone.

14. What to submit

Everything in a single file — one notebook, or one `.py` — containing both implementations and the comparison. Do not send several files.

1. **Part A code** — with the function signatures given above, working for any `layer_dims`.
2. **Part B code** — the PyTorch version, also depth-independent.
3. **Printed output:** cost per epoch for both, and final test accuracy for both.
4. **Depth independence:** run Part A once with `[784, 128, 64, 10]` and once with `[784, 256, 128, 64, 10]`, showing both train. You should not have edited any function between the two runs.
5. **The initialization experiment** (Section 3): loss after one epoch with He, with `* 0.01`, and with `* 1.0`. Two or three sentences on what you observed.
6. **The loss-curve plot** — both implementations on one figure — and the two training times.
7. **Written answers** to the five questions in the training-loop note above (the four mappings, plus `zero_grad`).
8. **Two to four sentences** on the first real bug you hit in Part A and how you found it. Be specific — "my db had shape (128, 64) because I forgot `keepdims=True`" is an answer; "I had some shape errors" is not.